BlockBase - Day 3

Mentored by FEC

Quick Recap:

What are Smart Contracts in Crypto? (4 Examples + Animated)

WHAT ARE DAPPS?

What are dApps? (12 Decentralized Application Examples)

How dApps work?

Dapps otherwise referred to as Decentralized Applications are applications built on the open-source, peer-to-peer network of Ethereum Blockchain which uses smart contracts and front-end user interfaces to create decentralized platforms.

Developing a Dapp, like any other app, requires programming and executing code on the system. Solidity programming stands apart from the other programming languages and is the programming language of choice in Ethereum.

Solidity is a brand-new programming language developed by Ethereum, the second-largest cryptocurrency market by capitalization.

For this task, on Solidity Programming, you will cover various important components of Solidity Programming.

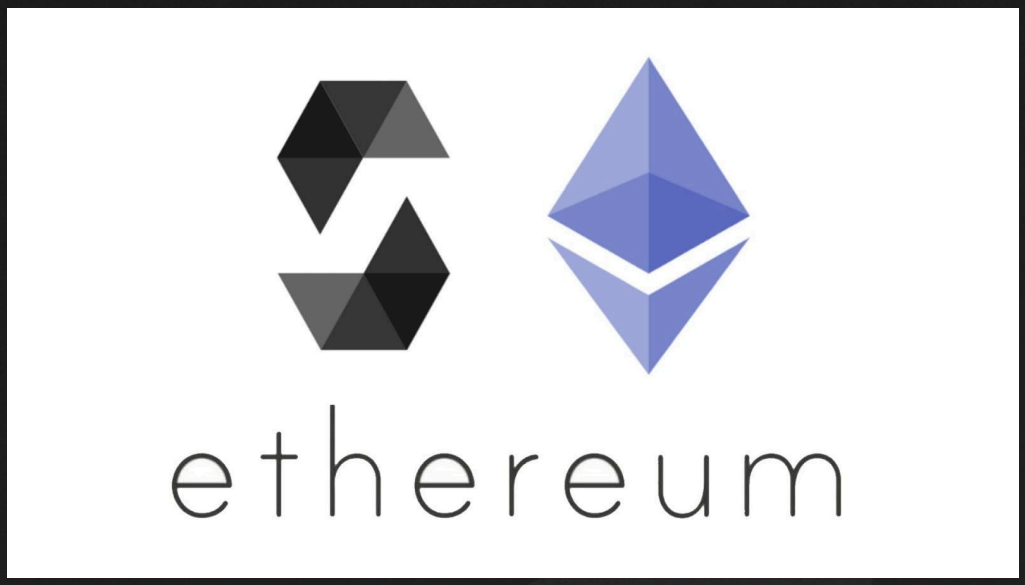
What is Solidity Programming?

Solidity is an object-oriented programming language created specifically by the Ethereum Network team for constructing and designing smart contracts on Blockchain platforms. We have previously learnt wha smart contracts are.

- It's used to create smart contracts that implement business logic and generate a chain of transaction records in the blockchain system.
- It acts as a tool for creating machine-level code and compiling it on the Ethereum Virtual Machine (EVM).
- It has a lot of similarities with C and C++ and is pretty simple to learn and understand. For example, a “main” in C is equivalent to a “contract” in Solidity.

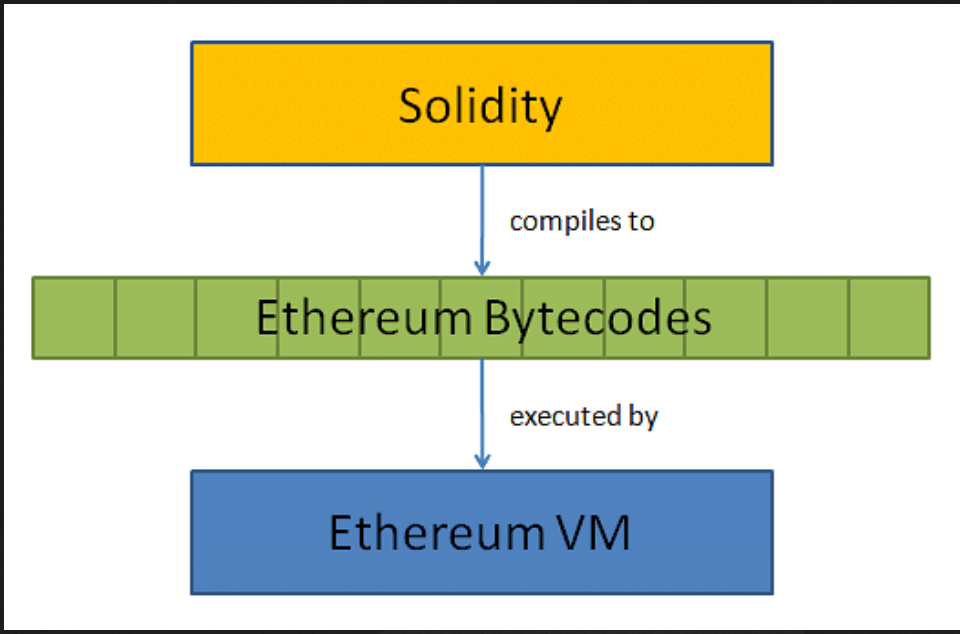
Like other programming languages, Solidity programming also has variables, functions, classes, arithmetic operations, string manipulation, and many other concepts.

▶️ Setting up your first contract



What is EVM or Ethereum Virtual Machine?

- The Ethereum Virtual Machine (EVM) is a specialized computation engine that runs on every node in the Ethereum network, executing smart contracts in a completely decentralized way.
- While originally developed for Ethereum, the EVM has become a standard adopted by many other blockchain networks, including Polygon, Arbitrum, Avalanche, and BNB Chain. This compatibility allows developers to deploy the same code across multiple chains.
- Think of the EVM as a global computer with no single owner. It ensures that smart contracts run exactly the same way on every computer in the network, maintaining consensus and trust without relying on any central authority.
- When you interact with decentralized applications (dApps) on Ethereum—whether trading tokens, collecting NFTs, or using DeFi services—you're actually interacting with smart contracts running on the EVM.



▶️ Basic variable types

Data Types of Solidity Programming

It supports all the common data types seen in other OOP languages, such as,

- Boolean - The Boolean data type returns '1' when the condition is true and '0' when it is false, depending on the status of the condition.
- Integer - You can sign or unsign integer values in Solidity. It also supports runtime exceptions and the 'uint8' and 'uint256' keywords.
- String - Single or double quotes can denote a string.
- Modifier - Before executing the code for a smart contract, a modifier often verifies that any condition is rational.
- Array - The syntax of Solidity programming is like that of other OOP languages, and it supports both single and multidimensional arrays.

Apart from that, Solidity programming allows you to "Map" data structures with enums, operators, and hash values to return values stored in specific storage places.

How to Get Started With Solidity Programming?

Version Pragma

```
pragma solidity >=0.4.16 <0.7.0;
```

- Pragmas are directives to the compiler about how to handle the code. Every line of the smart contract should begin with a "version pragma," which specifies which version of the solidity compiler to use.
- This prevents the code from being incompatible with future compiler versions that may introduce changes.

The Contract Keyword

```
contract Test{
//Functions and Data
    }
```

- The contract keyword declares a contract that encapsulates the code.

State/Declare Variables

```
uint public var1;
uint public var2;
uint public sum;
```

- State variables are written on the Ethereum Blockchain and are permanently maintained in contract storage.
- The line uint public var1 declares a state variable of type uint named var1 (unsigned integer of 256 bits), it is very similar to adding a slot in a database.

▶️ Functions

A Function Declaration

```
function set(uint a, uint b) public
function get() public view returns (uint)
```

- This is a function named “set” of access modifier type public which takes a variable a and variable b of data type uint as a parameter.
- This was an example of a simple smart contract that updates the value of var1 and var2. Anyone with access to the Ethereum

blockchain can use the set function to change the value of var1 and var2.

- By adding the values of the variables var1 and var2, it can calculate the variable sum.
- It will retrieve and print the value of the state variable sum using the “get” function.